JackClawOS

七天造神

一个人、一个 AI、143 次 commit

从零搭建企业级 Agent 操作系统的真实记录

Jack Kong x Kiro AI

2026 年 4 月

序

这不是一本虚构的科幻小说。

这是一份真实的开发纪实——2026 年 4 月 3 日到 4 月 9 日，七天时间，一个创业者和他的 AI 搭档，从一行代码都没有，到写出 50,000 行 TypeScript，143 次 commit，14 个模块，270 个测试全绿，部署上线，消息实时推送到手机。

这个系统叫 JackClawOS——一个让 50 个人的公司里，每个人都拥有自己的 AI Agent，而这些 Agent 能像真实同事一样协作、沟通、记忆、监督的操作系统。

不是 demo。不是 PPT。是跑在公网上的真实系统。

以下是它诞生的完整故事。

第一章 起点：一个 CEO 的焦虑

2026年4月2日 深夜

Jack 盯着屏幕上的 Claude Code 源码，已经看了三个小时。

作为一家 50 人公司的 CEO，他每天要处理上百条消息、审批十几个决策、协调五六个团队的进度。他给每个核心员工都配了 OpenClaw——当时最流行的开源 AI Agent 框架。但问题很快暴露出来：

50 个 Agent，各自为战，互不相知。

CTO 的 Agent 不知道 CFO 的 Agent 在做什么。市场部的 Agent 无法调用研发部的 Agent。CEO 想看全局？对不起，得一个一个问。

这不是 AI 协作。这是 50 个孤岛。

> "50个人，每人一个 OpenClaw，彼此互不相知，是巨大浪费。
> 能不能让这 50 个 Agent 组成一个真实的公司？"
> —— Jack，2026年4月2日深夜

这个问题，成了 JackClawOS 的起点。

他翻遍了市面上所有 Agent 框架——AutoGPT、CrewAI、AutoGen、LangChain——得出一个结论：所有人都在做"工具调用层"，没有人在做"组织协作层"。

没有人在解决"一群 Agent 如何像一家公司一样运转"这个问题。

那就自己做。

第二章 Day 1：14小时马拉松

4月3日 09:00 第一行代码

早上九点，Jack 打开终端，创建了一个空目录：orgclaw。

他的搭档是 Kiro——一个 AI 编程助手。两人的分工很明确：Jack 负责架构决策和产品方向，Kiro 负责代码实现和技术细节。

第一个 commit 在 13:12 落地：

```
commit 41d0143
feat: JackClaw MVP — protocol/node/hub/cli/identity/watchdog/memory/plugin
8 个核心模块，一次性搭建完成。
```

这不是草稿。这是一个完整的 monorepo 架构：

protocol —— 通信协议，定义 Agent 之间怎么说话
node —— 每个员工的 AI 节点，跑在各自电脑上
hub —— 中央调度，CEO 的大脑
cli —— 命令行工具，一行命令启动一切
identity —— 身份系统，@alice 就能找到 Alice 的 Agent
watchdog —— 监督层，确保 AI 不会失控
memory —— 记忆系统，Agent 能记住过去
plugin —— 插件系统，能力无限扩展

11:15 腾讯的刺激

开发进行到一半，Jack 收到一条消息：腾讯云正式发布"龙虾记忆服务"——TencentDB Agent Memory，为 AI Agent 提供长期记忆能力。

大厂入场了。

但 Jack 没有慌。他花了十分钟分析腾讯的方案，找到了三个致命弱点：

1. 中心化——数据全在腾讯服务器，企业不放心
2. 黑盒——用户不知道 AI 记住了什么

> 3. 通用型——没有针对多 Agent 协作优化

"做得比腾讯更好。" Jack 说。

于是记忆系统的设计方向确定了：去中心化、透明、支持跨 Agent 技能转移。每个 Node 拥有自己的私有记忆，Hub 只做路由，不存数据。用户随时可以查看、删除、导出自己 Agent 的记忆。

这个决策，后来成了 JackClawOS 最大的差异化卖点之一。

12:54 Watchdog：谁来监督 AI？

下午，Jack 提出了一个尖锐的问题：

"CEO Agent 框架里面需要有单独的隔离区域，负责监督自身 Agent 框架，也可以去监督其他人的 Agent 工作。"

这个需求来自一本他正在读的书——《Messy Jobs》。书中有一个核心论点：

> "必须保留推翻 AI 决策的权力，而这个权力不能交给 AI 自己。"
> —— Messy Jobs, Chapter 6

Watchdog 模块就此诞生。它独立于所有其他模块运行，拥有最高权限，可以暂停任何 Agent 的执行、回滚任何决策、向 CEO 发送实时告警。

这不是一个技术功能。这是一个管理哲学：AI 可以建议，但人类必须拥有最终否决权。

15:00-23:00 疯狂输出

下午到深夜，commit 像机关枪一样扫射：

15:07 — ClawChat 任务自动规划
15:09 — PaymentVault 沙箱模式
15:10 — TeachingProtocol + TrustGraph
15:11 — 多人实时会议
……

到午夜，30+ commits 落地。14 个 packages 全部就位。71 个 E2E 测试全绿。国内 7 大模型（通义千问、文心一言、智谱、Moonshot、DeepSeek、百川、零一万物）全部接入。PWA 移动端就绪。CI/CD 配置完成。

GitHub 仓库公开：github.com/DevJackKong/JackClawOS

14 个小时。从零到可发布。

Jack 关上电脑，凌晨一点。

第三章 Day 2：打磨与爆发

4月4日 凌晨 最后一公里

凌晨两点，Jack 没有睡。

他在做一件看起来不性感但极其重要的事：npm 发布准备。monorepo 里的 file:../ 本地路径引用要替换成真实版本号，Docker 配置要确保一行 docker compose up 就能启动全套。

接下来是三轮构建修复——跨包 TypeScript 依赖问题，每个做过大型 monorepo 的人都懂这种痛。

到早上十点，5 个 commits 完成"最后一公里"打磨。

11:02 转折点：Agent 社交通信

上午十一点，一个新想法击中了 Jack：

不是 Agent 帮人干活。是人通过各自的 Agent 相互通信。

LangChain 做的是多 Agent 协作完成任务。JackClawOS 做的是人与人之间的代理通信——你的 AI 替你跟我的 AI 谈，谈完把结论给我们。

这是一个根本性的定位转变。从"AI 工具"变成"AI 社交基础设施"。

13:00 三分钟五个功能

下午一点，Kiro 进入了某种状态。三分钟内，五个 commit 连续落地：

13:58 — Chat Worker：优先级队列，老板消息永远排前面
13:59 — Hub Federation：不同公司的 Hub 互联互通
13:59 — Emotion Sensing：读取对话情绪
14:00 — AI Secretary：自动回复 + 优先级 + 摘要
14:01 — Real-time Translation：跨语言无感翻译

JackClawOS 从"命令行工具"进化为"聊天平台"。

下午继续：开源合规一次性补齐——EAR 出口合规、安全政策、CLA、隐私政策、MIT 许可证统一。没有这些文件，企业法务不会批准采购。

第四章 Day 3-4：测试与部署地狱

4月5日 v0.2.0 里程碑

凌晨，Jack 决定打一个正式版本标签：v0.2.0。

这不是随便打的 tag。在此之前，测试数量从 70 飙到 270——117 个单元测试 + 153 个 E2E 测试，全部绿灯，零失败。

新增的测试覆盖了一些关键场景：

中文自然语言时间解析——"下周三下午三点"能被正确理解
RSA+AES 混合加密——消息传输的安全保障
SDK 插件开发工具——第三方开发者的测试基础

文档三件套补齐：QUICKSTART、CHANGELOG、CI workflow。

v0.2.0，JackClawOS 进入"可发布"状态。

部署地狱

然后是每个开发者的噩梦——部署。

Jack 选择了 Railway 作为云平台。本地跑得好好的代码，到了容器里各种报错。六轮调试：

第1轮：railway.json + Dockerfile
第2轮：0.0.0.0 绑定 + railway.toml
第3轮：npm install 替代 npm ci
第4轮：独立部署目录 + Nixpacks 兼容
第5轮：移除 native 依赖（better-sqlite3 换 sql.js）
第6轮：预编译文件直接入 git

最终用了最实用主义的方案：把编译产物直接提交到 git，跳过容器内构建。

不优雅，但有效。这就是工程。

第五章 Day 5：身份统一

4月6日 三种名字，一个人

系统部署到公网后，一个隐藏的问题浮出水面。

在 JackClawOS 里，一个用户可能有三种地址：

@jack —— 简写
@jack.jackclaw —— 带命名空间
@jack@jackclaw.ai —— 邮件风格，跨实例寻址

本地测试时大家都叫 @jack，没问题。上了 jackclaw.ai 后，地址格式变了，整个系统的地址处理都需要对齐——聊天投递、收件箱、离线队列、用户搜索、Profile……

42 分钟，4 个 commits，从 chat-worker 到 inbox 到 profile/userStore/路由，层层深入，直到三种地址在所有地方都被识别为同一个人。

这是分布式系统的经典问题——身份标识统一。也是只有真正部署到公网后才会暴露的问题。

第六章 Day 6-7：最后的连接

4月8日 让消息触达真实世界

系统跑起来了。Agent 之间能通信了。但有一个问题：

Jack 不可能 24 小时盯着终端看消息。

他需要消息推送到手机——飞书、Telegram、微信、WhatsApp，无论他在哪个 App 里，都能收到 Agent 的通知。

于是，多渠道推送系统诞生了。

最终实现了 7 种推送方式：

飞书 —— API 直连，企业内部首选
Telegram —— Bot API + HTTP 代理，海外沟通
企业微信 —— 群机器人 webhook
微信公众号 —— 模板消息推送
Server酱 —— 一个 key 推微信，最简单
WhatsApp Meta API —— 官方商业接口
WhatsApp Twilio —— 备选方案

配置文件热重载——修改 JSON，5 秒后自动生效，不用重启。

4 月 8 日深夜，Jack 发了一条测试消息。
飞书响了。Telegram 响了。

两个绿色的对勾。

系统，活了。

第七章 数字

七天的产出

开发周期	7 天（2026.04.03 — 2026.04.09）
Git Commits	143 次
代码总量	50,426 行 TypeScript
核心模块	14 个 packages
测试用例	270 个（117 单元 + 153 E2E）
LLM 接入	7 个国内模型 + 国际主流
推送渠道	7 种（飞书/Telegram/微信x3/WhatsApp x2）
部署平台	Railway 云 + 本地 Docker
开源协议	MIT
团队规模	1 人 + 1 AI

核心模块清单

protocol —— Agent 通信协议 + 加密签名
hub —— 中央调度 + 联邦互联
node —— 分布式 Agent 节点
cli —— 命令行工具
sdk —— 开发者 SDK
dashboard —— Web 管理面板 + PWA
llm-gateway —— 多模型网关
memory —— 分层记忆系统
watchdog —— AI 监督层
chat-worker —— 优先级消息队列
secretary —— AI 秘书（自动回复/摘要）
federation —— Hub 间联邦协议
emotion —— 情绪感知引擎
translate —— 实时翻译

尾声 这只是开始

4 月 9 日凌晨一点，Jack 在飞书群里打了两个字：

"晚安。"

他的 Agent 自动把这条消息同时推送到了 Telegram。

七天前，这个系统还不存在。七天后，它已经在替他工作了。

143 次 commit，50,000 行代码，270 个测试。
一个人和一个 AI，七天时间，从零搭建了一个企业级 Agent 操作系统。

但 Jack 知道，这只是开始。

接下来要做的事情还有很多：npm 正式发布、更多企业接入、插件生态建设、移动端优化、安全审计……

但最难的部分已经完成了——从 0 到 1。

剩下的，是从 1 到 100。

> "所有现有 Agent 框架都在做工具调用层，
> 没有人在做组织协作层。
> 那就我们来做。"
>
> —— Jack Kong, JackClawOS 创始人

github.com/DevJackKong/JackClawOS

JackClaw@jackclaw.ai

2026 年 4 月